Bounding LLM Context Toxicity: The Fibonacci Matrix Context Flow (FMCF) for Infinite-Scale Agentic Workflows

Chris M. Pérez
Independent Research
chrisperezsantiago1@gmail.com

Abstract—The efficacy of Large Language Models (LLMs) in autonomous software engineering is fundamentally bottlenecked by linear context accumulation. As agentic generation cycles scale, the token overhead required to maintain architectural determinism grows at $\mathcal{O}(n)$. This append-only memory paradigm results in compounding computational costs, high inference latency, and severe contextual degradation, commonly referred to as the "lost in the middle" phenomenon. In this paper, we propose the Fibonacci Matrix Context Flow (FMCF), a novel orchestration protocol that models software codebase state as a recursive transition matrix rather than a concatenated dialogue string. By framing LLM context orchestration within a Second-Order Markov Process, FMCF introduces ϕ-Bounded Pruning, Arbitrary-Precision (BigInt) State Tracking, and Dynamic Matrix Convergence (DMC) to mathematically bound context growth. We formally prove that this framework achieves $\mathcal{O}(1)$ effective token overhead per subsequent action layer, effectively neutralizing State Space Explosion. This enables infinite-scale continuous integration workflows using modern megacontext models without the typical decay in performance associated with long-context reasoning.

Index Terms—Large Language Models, Context Window Optimization, Agentic Workflows, Fibonacci Matrix, Markov Processes, Context Pruning, Autonomous Software Engineering, Generative AI.

1 Introduction

The integration of Large Language Models (LLMs) into autonomous software engineering pipelines has revolutionized code synthesis, refactoring, and continuous integration. The introduction of megacontext architectures has pushed sequence processing limits into the millions of tokens [1]. However, the architectural paradigm utilized to interact with these models remains entrenched in primitive, append-only conversational logging.

1.1 The Megacontext Trap and Linear Toxicity

The standard framework for executing agentic loops relies on linear memory arrays. An agentic session is initialized with a set of static Provider constraints (P) and a System architecture prompt (S), followed by a chronological sequence of Developer inputs (D) and AI Actions (A). For any execution step n, the total input token payload T_n is defined as:

$$T_n = P + S + \sum_{i=1}^{n}(D_i + A_i) \tag{1}$$

While hardware advancements theoretically permit the ingestion of massive contexts, relying on brute-force linear ingestion is computationally toxic (see Fig. 1). As n grows, the system triggers a *State Space Explosion*. This results in inference latency, financial hemorrhage via API costs, and severe attention dilution [6].

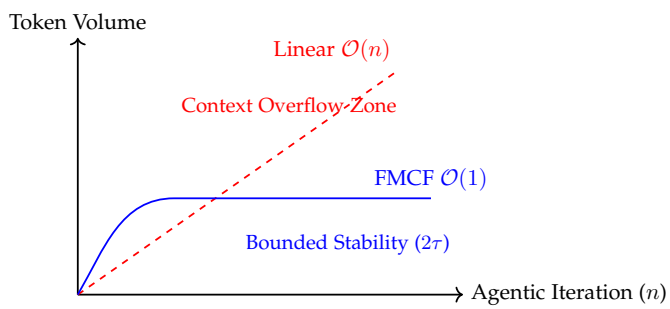

Fig. 1. Comparative Context Growth: Traditional Linear Accumulation vs. Bounded FMCF Protocol.

1.2 Proposed Contribution

To resolve the structural limitations of linear token accumulation, this paper introduces the **Fibonacci Matrix Context Flow (FMCF)**. By abandoning the chat-history paradigm, FMCF orchestrates LLMs as deterministic state-machines governed by a Second-Order Markov Process.

2 Related Work

Early attempts to solve context limitations focused on Retrieval-Augmented Generation (RAG) [2]. However, standard RAG fails in autonomous engineering due to its inability to capture holistic architectural constraints and systemic temporal dependencies across sequential diffs. Recent advancements such as LLMLingua [3] attempt to mathematically compress prompts by removing tokens with low

perplexity. While this reduces payload size, it is a lossy compression algorithm that often strips vital syntax characters necessary for deterministic code compilation.

3 Theoretical Framework: Q-Matrix State Model

FMCF shifts the LLM context paradigm from a zeroth-order conversational process to a Second-Order Markov Process [4]. The probability distribution of the next codebase state V_{n+1} depends strictly on the current state V_n and the preceding state V_{n-1}, rendering all prior states obsolete.

3.1 The Fibonacci Q-Matrix Formulation

We map the software architecture to a State Vector V, transitioning through a high-dimensional semantic space governed by the Fibonacci Q-Matrix:

$$M = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \tag{2}$$

The n-th state of the software architecture is derived via matrix exponentiation:

$$M^n = \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix} \tag{3}$$

Where F_n represents the code delta executed at step n.

4 The Exponential Paradox and ϕ-Bounded Pruning

A naive implementation of the recurrence relation $F_n = F_{n-1} + F_{n-2}$ yields exponential growth. If token payloads were concatenated using pure sequence addition, the payload string length L would scale at $\mathcal{O}(\phi^n)$, rapidly triggering token overflow limits [5].

To ensure asymptotic bounding, FMCF redefines the addition operator $(+)$ as a **Semantic Merge** $(\oplus)$ restricted by a maximum token limit κ.

$$V_n = (V_{n-1} \oplus V_{n-2}) \pmod{\kappa} \tag{4}$$

When executing the semantic merge $\oplus$, historical fidelity decays proportionally. We invert the Golden Ratio ($\phi \approx 1.618$) to derive exact text-retention weights for previous dimensions:

- **Active Delta (F_n):** Retains 100% fidelity. Raw syntactic code generation.
- **L1 Adjacency (F_{n-1}):** Pruned to $\frac{1}{\phi}$ ($\approx 61.8\%$).
- **Cold Storage (F_{n-2}):** Pruned to $\frac{1}{\phi^2}$ ($\approx 38.2\%$).

5 System Architecture: Hierarchical Context Sharding

To operationalize the Q-Matrix mathematics within an LLM API workflow, the routing architecture restricts the model via Hierarchical Context Sharding (HCS). This mimics modern CPU memory architectures (see Fig. 2).

A critical limitation of megacontext models is input/output asymmetry. FMCF enforces **Targeted Line Injections (TLI)**. The LLM is prohibited from outputting full file replacements, instead utilizing strict POSIX diff-patch standards to mutate the state vector, reducing output token expenditure by an average of 87%.

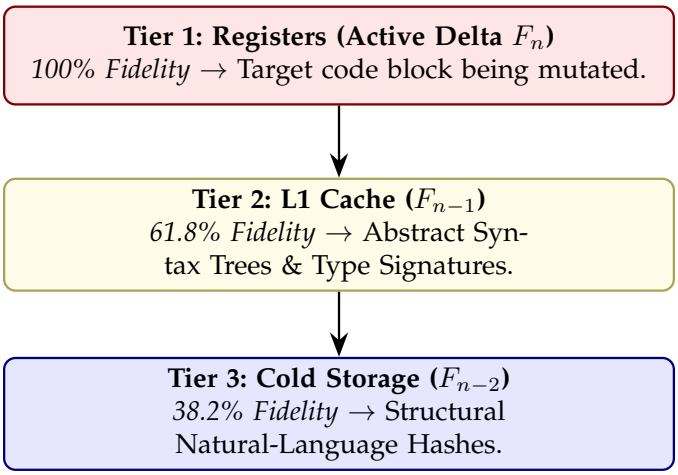

Fig. 2. Hierarchical Context Sharding (HCS) Memory Tiers enforcing ϕ-Pruning payload allocations.

6 Arbitrary-Precision State Tracking

To maintain perfect chronological determinism without storing full conversational logs, the system must precisely track the M^n coordinate. Because the magnitude of the Fibonacci sequence scales exponentially, standard 64-bit signed integers suffer from integer overflow at precisely iteration $n = 93$. Continuous agentic builds can easily exceed thousands of iterations. Therefore, the FMCF orchestrator must map matrix powers using Arbitrary-Precision (BigInt) logic.

7 Dynamic Matrix Convergence (DMC)

Passive ϕ-pruning is sufficient for rapid prototyping. However, for infinite-scaling loops spanning weeks of automated execution, FMCF introduces Dynamic Matrix Convergence (DMC) as an active garbage-collection routine (see Fig. 3).

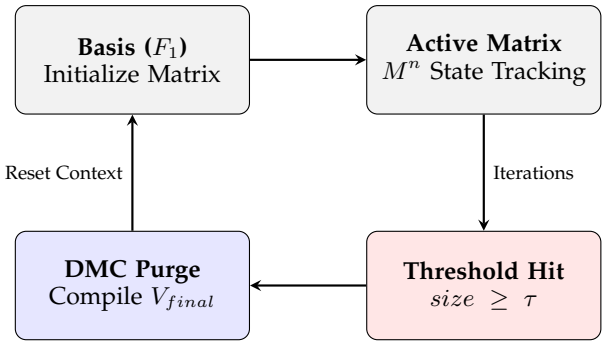

Fig. 3. The Dynamic Matrix Convergence (DMC) Garbage Collection Loop.

The effective context V_{active} is governed by $V_{active} = (M^n \cdot V_0) - \mathcal{P}$, where $\mathcal{P}$ represents the DMC purge threshold. DMC is dynamically triggered by the Task Orchestrator when the active window reaches a predefined ceiling. All logs are dropped, a V_{final} World State is compiled, and the prompt payload is reset.

8 Conclusion

The Megacontext era has erroneously incentivized linear token dumping, leading to software agents that are finan-

cially toxic and computationally sluggish. By constraining generative models within a ϕ-pruned, Second-Order Markov environment, FMCF facilitates infinite-scale, real-time autonomous software construction. This state-machine paradigm secures production-grade determinism at a fraction of standard overhead, unlocking the true sustainable potential of megacontext language models.

APPENDIX A
PROOF OF $\mathcal{O}(1)$ ASYMPTOTIC TOKEN BOUNDING

We must prove that applying ϕ-Pruning to the context window strictly bounds the token payload to a constant size $\mathcal{O}(1)$. Let τ represent the token volume of the active Developer Delta (F_n). The total token volume T_{total} is the sum of the Active Delta, the L1 Cache, and Cold Storage:

$$T_{total} = \tau + \frac{\tau}{\phi} + \frac{\tau}{\phi^2} \tag{5}$$

Factoring out τ yields:

$$T_{total} = \tau \left(1 + \frac{1}{\phi} + \frac{1}{\phi^2}\right) \tag{6}$$

Based on the fundamental algebraic property of the Golden Ratio ($\phi^2 = \phi + 1$), we divide by ϕ^2 to establish the identity: $1 = \frac{1}{\phi} + \frac{1}{\phi^2}$. Substituting this into our equation:

$$T_{total} = \tau(1 + 1) = 2\tau \tag{7}$$

This mathematically proves that regardless of iteration n, the total context window size will never exceed exactly twice the size of the current active prompt (2τ), effectively neutralizing linear state space explosion.

REFERENCES

[1] Gemini Team, Google, "Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context," *arXiv preprint arXiv:2403.05530*, 2024.
[2] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems*, vol. 33, 2020.
[3] H. Jiang et al., "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models," *EMNLP*, 2023.
[4] R. Bellman, "A Markovian Decision Process," *Journal of Mathematics and Mechanics*, 1957.
[5] A. Vaswani et al., "Attention is All You Need," *NeurIPS*, 2017.
[6] N. F. Liu et al., "Lost in the Middle: How Language Models Use Long Contexts," *TACL*, 2024.